Seamless Tech Onetap Away

Date : August -09-2025

GoLang Developer Challenge — Salary Advance Loan Service

Objective

Build a backend service in Go for a Salary Advance Loan system.
The focus is on secure authentication/authorization, data validation, and customer transaction rating.
You will be given JSON input files — assume they are already received by the service.

Files Provided

1. **Customer List** (`customers.json`)
   - Contains official bank records:
     `name`, `account_number`, `account_type`, `balance`, `image` (optional)
2. **Transaction History** (`transactions.json`)
   - May contain transactions for multiple accounts.
   - Some customers may have **no transactions**.
3. **50-Customer Sample File** (`sample_customers.json`)
   - Data from an external source.
   - **Two records contain intentional errors**:
     - One has an **account number type/format error**
     - One has a **name mismatch** compared to the canonical list

From the provided customer records, split them into five batches:

- Three batches (each containing 10 records) should be fully valid — all names and account numbers match the customer list, and all pass verification.
- Two batches (each containing 10 records) should contain intentional issues:
  - One batch should have one record with an invalid name (name does not match canonical list).
  - The other batch should have one record with an invalid account number (wrong type or incorrect format).

- Even for the batches with errors, all records (including the incorrect ones) should be saved after processing, but the incorrect ones must be marked as `verified=false` in the validation log.

Requirements

1. Authentication & Authorization

- Implement **JWT-based authentication**
- **Store passwords securely**
- **Two roles:**
  - `admin`
  - `uploader`
- Only authenticated users can process JSON data.
- Add rate-limiting for login attempts.
- Document all security measures in the README.

2. Data Validation

Write a function to:

1. **Read** the three JSON files.
2. For each record in `sample_customers.json`:
   - Compare with the canonical customer list:
     - If `account_number` is invalid format/type → mark `verified=false`.
     - If `name` does not match (case-insensitive, trimmed) → mark `verified=false`.
     - If both pass → `verified=true`.
3. Produce a **validation log** per record:
   - `record_index`
   - `verified` (true/false)

    - `errors` (list of issues)
    - `normalized_record` (if verified)

4. Store only records that pass verification in the database (or in-memory storage). For any unverified batch, highlight it in the validation log by adding extra details, and explicitly state the reason for failure — for example, `"name does not match"` or `"account number does not match"`.

3. Transaction Generation & Rating

- From `transactions.json`, map transactions to each customer.
- If a customer has no transactions, **generate synthetic ones**:
    - Ensure transactions do not create a negative balance unless overdrafts are allowed.
    - Document generation logic.
- **Calculate customer rating** (1–10) based on:
    - Number of transactions
    - Total transaction volume
    - Duration between first & last transaction
    - Balance stability during transactions
- **Example formula** (you can adjust):
  ```ini
  CopyEdit

  rating = weighted_sum(count_score, volume_score, duration_score, stability_score)
  ```
-
- Return rating for each customer along with a breakdown of how it was calculated.

Expected Deliverables

- **Source Code** (Go)
    - Modular, well-structured, idiomatic Go
    - Uses Go modules

    - Includes basic unit tests for:
        - Authentication
        - Validation logic (should detect the two faulty records)
        - Rating calculation
- **README** (clear instructions)
    - How to run the service
    - How validation is done
    - How rating is calculated
    - Security measures used
- **Validation Output** (log/report)
    - JSON or table format
    - Shows which records passed/failed validation and why

Evaluation Criteria

Area	Weight
Code Structure & Readability	30%
Validation Accuracy	25%
Security Implementation	20%
Transaction & Rating Logic	15%
Documentation & Tests	10%

We expect you to submit your completed challenge by **August 18, end of day**.
At any time before then, you may send us a list of questions via Google Form or email with all relevant details.
If you need clarification, we can arrange a **one-hour briefing** starting from Monday at your convenience.
We wish you the best of luck! Our Tech Lead, **Alazar**, will be available to provide hints, best practices, or any resources you may require.

+251111400810
info@onetap.et
Onetap.et